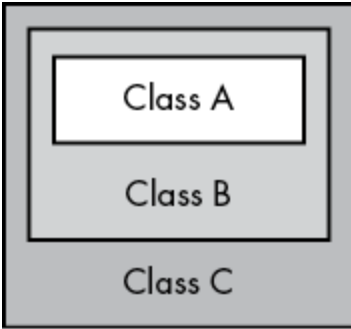


all the methods and instance variables in class `A`. This type of hierarchy can be very useful when building more and more specific classes. Class `A` can be very general, class `B` more detailed, and class `C` even more specific.

[Figure 10-10](#) provides a different way to think about the relationships in a class hierarchy.



[Figure 10-10](#): A different way of portraying a class hierarchy

Here, the client sees only class `C`, but this class is made up of all the methods and instance variables defined collectively in classes `C`, `B`, and `A`.

The `pygwidgets` package uses a class hierarchy for all widgets. The first class in `pygwidgets` is the abstract class `PygWidget`, which supplies basic functionality to all widgets in the package. Its code consists of methods that allow for showing and hiding, enabling and disabling, getting and setting the location, and getting the nickname (internal name) of any widget.

There are other classes in `pygwidgets` that are used as abstract classes, including the aforementioned `PygWidgetsButton`, which is the base class of both `TextButton` and `CustomButton`. [Figure 10-11](#) should help make this relationship clear.